LETMA 2026 Summer School
Machine Learning on Geophysical Data — Hands-on Labs

LETMA 2026 Summer School

Overview

Three labs, each a complete *question* → *hypotheses* → *data* → *network* → *loss* → *evaluation* loop on geophysical data:

Lab	Method	Data	Failure mode studi
1. FNO for Burgers	Fourier Neural Operator	1-D viscous Burgers	the loss landscape and
2. FFNO for infrasound	Factorized FNO	boundary-layer atmospheres	ML pathologies (seeds
3. GNN for a sensor network	Graph Neural Network	synthetic seismic picks	oversmoothing

How to work. Each lab runs top to bottom. Every exercise has an automatic checker: if your attempt fails, the reference implementation takes over and *the lab continues* — an exercise can cost you minutes, never the lab. Every assumption is stated as a numbered hypothesis (H1, H2, ...) *before* being used; the mathematical notes carry the rigorous statements and are optional reading. Each notebook exists in a `*_student` version (you run it) and a fully executed `*_solution` version (every figure and animation already in it).

Lab 1 — Fourier Neural Operator for the 1-D Burgers equation

Scientific question. *Can a network learn the* solution operator *of a PDE — the map* $G^\dagger : u_0 \mapsto u(\cdot, T)$ *— and what do the training loss and its hyperparameters do to what is learned?*

What will be done.

1. **The problem, with every hypothesis (H1–H7).** The time-T map of viscous Burgers on $[0, 1)$; inputs from a Gaussian random field; data distributions and animated PDE dynamics.
2. **The one new idea: spectral convolution.** You check the spectral decay of the data (H7) and the mode-truncation experiment, then implement the spectral convolution yourself.
3. **Assemble the FNO and train it.** Lift → spectral blocks → project; training is animated prediction-by-prediction on unseen fields.
4. **The loss, its hyperparameters, its landscape.** Loss-family and weight-decay sweeps; the *animated optimizer race* on a 2-D slice of the true loss landscape, run to the trained loss L^*; the two-minima experiment; the hyperparameter corner plot.
5. **Evaluate, super-resolve, transfer.** Test error, and the zero-shot evaluation of the model trained at N=64 directly at N=256.

Exercise F1 (checker + fallback). Implement the spectral convolution $(Kv)(x) = \sum_{k<m} R[k]\, \hat{v}[k]\, e^{2\pi i k x}$: one real FFT, a mode-by-mode complex matrix product, one inverse real FFT. *Question to answer:* why does keeping m=16 modes lose almost nothing on this data?

Question (forward pass). The notebook assembles the full forward pass $a(x) \to P \to$ blocks $\to Q \to u(x)$, $v_{\ell+1} = \mathrm{GELU}(K_\ell v_\ell + W_\ell v_\ell)$: where is the computation happening — in physical space or in the learned latent space of width 32?

Questions the experiments answer. Does Adam reach the trained loss faster than SGD with momentum (and by what factor)? Do two random seeds end in the same minimum? Which hyperparameter (modes, width, layers) buys the most accuracy at fixed budget? Does the operator trained at N=64 work at N=256 without retraining — and why?

Lab 2 — Factorized FNO for infrasound boundary-layer propagation

Scientific question. *A few ground receivers record infrasound from an explosion at 2–11 km range. The signals were shaped by the effective sound-speed profile $c_{\mathrm{eff}}(z)$ of the atmospheric boundary layer ($z \in [0, 2]$ km) — can a network predict the recorded waveforms directly from that profile, and can we keep such an operator affordable?*

What will be done.

1. **The physics, with every hypothesis (H1–H13).** A transparent parabolic-equation solver; an *animated wavefield* for one atmosphere; ray tracing and a hodograph, with the input profile shown vertically (altitude on the y-axis, celerity on the x-axis) and the recorded traces at their true arrival times.
2. **The learning problem.** 256 synthetic atmospheres (200/28/28 split); profile in, *four* ground traces out, predicted jointly in one evaluation.
3. **The FFNO.** The factorized spectral layer $(Kv)^\wedge = R_x[k_x]\hat{v} + R_y[k_y]\hat{v}$ (tensor rank-1 constraint, H13); training with the waveform-emerging animation; where the computation happens: the learned latent space between the profile encoder and the waveform readout.
4. **The loss, its landscape, the optimizer race.** Loss family and weight decay on this task; the race to the trained loss on the true landscape slice.
5. **Overfitting under small data:** the same model trained on eight atmospheres only — the training/validation split is the instrument that catches it.
6. **Evaluate on unseen atmospheres:** held-out profiles and all four receivers.

Exercise F2 (checker + fallback). Implement `signal_spectrum` (two NumPy calls) and check H4/H13 yourself: where does the energy of a recorded trace live in frequency, and how many Fourier time modes cover the band?

Lab 3 — Graph Neural Networks: learning on an irregular sensor network

Scientific question. *A seismic network is a handful of stations scattered over a map — not a grid, not always the same stations, in no meaningful order. How do we make a network respect that geometry, remain correct when stations drop out, and avoid the failure mode that plagues deep graph models: oversmoothing?*

What will be done.

1. **How the graph is constructed.** Nodes = the stations that recorded the event (a random subset, 0–30% down, H3); edges = k-nearest neighbours (k=6); features = relative positions and arrival times (H6); the *animated k-NN construction*, node by node.

2. **Message passing, defined precisely.** $m_{ij} = M_\theta(h_i, h_j, p_j - p_i, \|p_j - p_i\|)$, $a_i = \frac{1}{|\mathcal{N}(i)|}\sum_j m_{ij}$, $h_i \leftarrow h_i + U_\theta(h_i, a_i)$; the hop-by-hop information-spreading animation.
3. **Oversmoothing, with depth.** The spectrum of the smoothing operator $P = D^{-1}A$: each mode decays at its own geometric rate $(1 - \lambda_k)^\ell$, only the mean survives; the three-panel animation (station colors, the state of *every* node collapsing to the mean, the Dirichlet energy); pure averaging vs. our residual layer; the depth experiment (train 1, 2, 4, 8, 16 layers).
4. **Train the locator** and watch *one* held-out epicentre converge to the truth, epoch by epoch.
5. **Evaluate in kilometres**, against a graph-free MLP baseline and as a function of the azimuthal gap.

Exercise G1 (checker + fallback). Implement the message and the forward pass of one message-passing layer (mean aggregation is provided by the base class).

Exercise G2 (checker + fallback). Assemble the full locator: encode → 4 layers → mean-pool → decode the hypocentre.

Questions the experiments answer. Why must the aggregation be a mean/sum (permutation invariance)? Why does the model still work when 8 of 30 stations are down, while a fixed-size vector MLP pays for it? At what depth do node states become interchangeable, and how do you diagnose it with one scalar (the Dirichlet energy)?

Optional extensions (after class)

- **Darcy scale-up:** the same operator philosophy on a 2-D elliptic PDE — a small factorized FNO2d, 200 epochs on a 2304-pair database shipped with the repo.
- **Labs 1 and 3 reimplemented in JAX** (Equinox + Optax): the same science, the functional programming model (`vmap`, `jit`, pytree loss landscapes, padded-and-masked graph batching).

Practicalities

Notebooks run on the Hedy cluster (a MIG slice of an H100 is enough) or on any laptop CPU; no internet is needed and all data are generated by the shipped solvers (or shipped as arrays for the JAX extensions). The executed `*_solution` notebooks contain every animation and figure.

On Hedy: one hop, the GPU, and Euporie

```
ssh -Y hedy # the single hop (-Y: X11 forwarding for live plots)

# GPU session on a MIG slice: ONE command allocates the slice, forwards X11
# and drops you into an interactive shell ON the GPU node:
salloc \
    --account=m26266 \
    --partition=gpu \
    --reservation=test_mig_letma \
    --nodelist=m-gpu[07-08] \
    --gres=gpu:nvidia_h100_80gb_hbm3_1g.20gb:1 \
    --nodes=1 \
    --ntasks=1 \
    --cpus-per-task=8 \
    --time=01:00:00 \
    bash -c '
        NODE=$(scontrol show hostnames "$SLURM_JOB_NODELIST" | head -n 1)
        echo "Allocated job: $SLURM_JOB_ID on $NODE"
```

```
       exec ssh -Y -C -t "$NODE" \
           "srun --jobid=$SLURM_JOB_ID --overlap --ntasks=1 --pty bash -i"
    '
# release afterwards with: scancel <jobid>

# then, ON the GPU node: first CHECK YOUR SLICE - one MIG device with ~20 GB:
nvidia-smi
# MIG devices:
# GPU GI CI MIG | Shared Memory-Usage
# 0 3 0 0 | 9MiB / 20096MiB <- your 1g.20gb slice
# (if the MIG section is absent or shows another size, exit and resubmit the salloc)

source /home/projects/m26266/letma2026-ml-labs/env/fno-gnn/bin/activate
# (or your own venv: python3 -m venv ~/envs/fno-gnn && source ~/envs/fno-gnn/bin/
    activate
# pip install -r /home/projects/m26266/letma2026-ml-labs/requirements.txt)
rsync -a --exclude env /home/projects/m26266/letma2026-ml-labs/ ~/letma2026-ml-labs/

cd ~/letma2026-ml-labs/notebooks/labs
# Hedy has no SSH port forwarding, so there is no browser interface on the cluster:
# euporie is the notebook interface in the terminal (on your own machine, jupyter lab
# works normally after a git clone + pip install).
euporie-notebook --kernel-name python3 lab_fno_student.ipynb
# optional, with the -Y forwarding: pop-up matplotlib windows instead of inline plots
LAB_PLOT_MODE=tk euporie-notebook --kernel-name python3 lab_fno_student.ipynb
# (tk needs a WORKING X display: the labs fall back to inline plots automatically)
```

Launch time: about 5 seconds on a warmed node, 1–2 minutes on a cold one. Everything (euporie, jupyter, torch, numpy, matplotlib, ...) is paged in from the cluster's shared filesystem on the first launch of a node; the screen shows `Loading...` / `No Kernel` meanwhile — this is not a freeze. The nodes are warmed up once before the session, so your launch should be quick: wait until the kernel badge `Python 3 (ipykernel)` appears in the bottom-right status bar, then the notebook is fully interactive. The `Rendering... (61%)` indicator may stay at a partial value: it counts the whole notebook, and cells below the visible area are rendered lazily when you scroll.

Euporie quick reference (verified against the installed version; the `python3` kernel is the activated venv's interpreter; the JAX extensions use the sibling venv `env/jax-hedy`):

Navigate	`Up/Down` (or `j/k`) select cells; `Home/End`; `PgUp/PgDn` (±5 cells)
Run	`Ctrl+Enter` run cell; `Shift+Enter` run and select next; `Alt+Enter` run and insert below
Edit	`Enter` enter edit mode; `Esc` leave; `Ctrl+\` split cell; `M` merge cells
Cells	`a/b` insert above/below; `d d` delete; `z` undelete; `x/c/v` cut/copy/paste
Cell type	`m` to markdown; `y` to code
Move	`Alt+Up/Alt+Down` move cells
Outputs	`w` expand/collapse; `W` wrap; `l` line numbers
Kernel	`i i` interrupt; `0 0` restart
File	`Ctrl+S` save; `Alt+S` save as; `Ctrl+O` open; `Ctrl+W` close tab
Quit	`Ctrl+Q` (prompts to save unsaved changes)

`Tab` cycles the focus between the notebook, the side pane and the command bar at the bottom (where less common actions, e.g. `run-all-cells`, can be typed). Figures render inline — terminal graphics on Kitty/Sixel-capable terminals, character previews elsewhere.